

Part III

GRPO Deep Dive

Why do we need reinforced learning and not supervised learning? Supervised learning is learning via mimicking and not via objective. You don't learn talking by imitation because you need to be scored at some point. Learn by imitation, then fine-tune by objective.

8 Actor-Critic Models vs Actor Models

The actor tries to solve the problem and the critic says how well the actor does. In PPO, the actor is the policy model and the critic is the value model. If GRPO doesn't have a value model, then who is the critic? Text is easy to generate, but the generated neighbors of a sentence are friendly because correct responses are surrounded by similar responses, e.g., "today I'm doing fine" is similar to "today I'm doing okay". This also makes it hard to criticize because subjectivity plays a strong role. However, math is hard to generate because the neighbors are unfriendly since correct responses are surrounded by wrong responses, e.g., " $2 + 2 = 4$ " is correct but " $2 + 2 = 5$ " is wrong, and this is very easy to check. In GRPO, the critic stays the same because it's frozen.

Definition 8.1. Actor-Critic Model is a model in which the Actor and Critic are updated during training.

Definition 8.2. Actor Model is a model in which the Actor is updated during training and the Critic is frozen during training.

9 Calculating advantage (per model)

How do we build a model that does math? Assume we have a space that contains all models and there is a perfect model in there. Supervised learning (modeling with labeled data) helps us get to the vicinity, but we have no more data so we have to rely on reinforced learning to proceed. We have to trust the model is correct at this point to proceed. We use a reward function (that serves like gradient descent) to push us in the best direction of our target. This is not supervised because we don't have a labeled score; the data has to be generated as you go.

Let Q be a dataset of questions, let q be a question sampled from Q with probability $P(Q)$, and o be the model's output to that question. How do we pick the questions? For the math example, in order to have the model to actually learn from the question, you need to generate math problems that weren't too easy (all responses are correct) or too hard (all responses are wrong). It needs a contrast to learn from. You can use existing RL data and synthesize additional training questions and find out their answers. It is common to assign difficulty because the model will not be able to solve the hard problems from the start, so you stage the difficulty. You can seed problems. One common way of evaluating difficulty by sampling n outputs, and then calculate the percentage correct.

We "ask" (sample) a question q and generate many (G) answers o_i for $i = 1 \dots G$. Let π be a policy function, which is a probability distribution over actions (individual tokens in our case), θ be the policy model parameters, q be a question sampled from a dataset of questions $P(Q)$

$$q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)$$

9.1 Scoring raw answers

We assign raw scores to answers iteratively. For example, solving $2x + 4 = 10$ has several steps we can score and be creative with the correctness criteria. You can parse the steps with an LLM to classify the ideas presented in each step and send them to the correct spots. It can be complicated because coming up with the correct tools and scoring criteria is not easy, but the reward models only need to be programmed to be hidden critics and not trained. For code and math, scoring by deterministic format and accuracy results goes a long way in improving models. It is aided by heuristic/frozen reward models for non-math/code problems.

For example, you could give an LLM a scoring rubric, and ask it to return the scores in some parseable way like a json object. To prevent hallucinations, it's definitely an additional engineering challenge to leverage the judge's responses. The judge might violate the format, skip parts of the metric, etc. You can either try to be robust to those issues, or just give up on those samples. If you're using an API to access a frontier model as the judge, hopefully it's pretty good, though?

9.2 Calculating rewards

In this specific lecture, we set the score as the reward, i.e., $s_i = r_i$ for $i = 1, 2, \dots, G$.

9.3 Calculating the advantage via normalization

Hard questions are more likely to get a low scored results, e.g., 0.01 for a strong attempt but completely wrong answer and 0 for a low-effort and just as wrong answer, which means model will have an optimization bias towards the easy questions. We'll have to normalize the scores/rewards to average 0 and variance 1 by mean-centering the scores, and then dividing by the standard deviation. In GRPO, the advantage for a given reward r_i is

$$A_i = \frac{r_i - \overline{\{r_i\}_{i=1}^G}}{\text{std}\left(\{r_i\}_{i=1}^G\right)}$$

Remark 9.1. This has a simple flaw. If every single one of the G sampled outputs o_i receives the exact same reward, then

$$\text{std}\left(\{r_i\}_{i=1}^G\right) = \frac{1}{G} \sum_{j=i}^G (r_j - \bar{r}_G)^2 = \frac{1}{G} \sum_{j=i}^G (r - r)^2 = 0$$

The solution is to add a very tiny number in the denominator, i.e.,

$$A_i = \frac{r_i - \overline{\{r_i\}_{i=1}^G}}{\text{std}\left(\{r_i\}_{i=1}^G\right) + \varepsilon}$$

10 Objective Function

10.1 Moving away from critic models

If we ask every candidate model to generate potential responses to score, then the outputs could be wrong but also technically be the best of the worst, so it's chaotic and lacks consistency. Generating many responses at this dimensionality is also computationally expensive. This is why we want to move away from the critic network. The solution is to only ask the base model the question q and only use its outputs $\{o_i\}_{i=1}^G$, i.e., given a policy model π with parameters θ_{old} ,

$$\{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(\cdot|q)$$

We calculate the outputs' advantages $\{A_i\}_{i=1}^G$, and these outputs can also be independently rated because they have probabilities associated with them (ala, softmax), i.e., $\pi_{\theta}(o_i|q)$ or traditionally the log probability $\log \pi_{\theta}(o_i|q)$. This means we can be updating our weights for model in the direction that likes the good answers more, even if they are not technically correct. Finally, we calculate the inner product of these two vectors to get the candidate score, i.e.,

$$\langle \pi_{\theta}(o_i|q), A_i \rangle$$

This is a continuous solution because small changes have small outputs, so we can perform gradient descent on this inner product. However, this is still very unstable.

10.2 Dealing with rarely formatted, but correct, responses

We do not want to punish eloquence due to rarity, e.g., “It is 4” vs “Evidently, it’s 4”. The probabilities the candidate models assign to eloquent answers could be low, but their output probabilities $\log \pi_\theta(o_i|q)$ should scale wrt the scores A_i in the base model. So we take probability of candidate divided by probability of base model and scale by the score, i.e.,

$$L_{sum}(\theta) = \left\langle \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, A_i \right\rangle$$

Since our goal is to calculate the expected loss, we would like to have an unbiased estimator, i.e.,

$$L_{mean}(\theta) = \frac{1}{G} \left\langle \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, A_i \right\rangle = \hat{L}(\theta)$$

where $\hat{L}$ is the sample mean, but stability and safety are more important in reinforcement learning.

Remark 10.1. The rewards r_i are sampled iid from the old policy model, but the advantages aren’t iid because they depend on each o_i . The sample variance (and sample standard deviation) is a nonlinear function of the samples, but magic happens if we sample enough times to get into the law of large numbers territory.

Proposition 10.1. *Let $\hat{L}(\theta)$ be the GRPO estimator sampled over a group of size G for a given prompt q , i.e.,*

$$\hat{L}(\theta) = \frac{1}{G} \left\langle \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, A_i \right\rangle$$

where $A_i = \frac{r_i - \overline{\{r_i\}_{i=1}^G}}{\text{std}(\{r_i\}_{i=1}^G) + \varepsilon}$ represents the group-relative advantage. For a finite size G , $\hat{L}(\theta)$ is a biased estimator of the true expected policy objective, but asymptotically unbiased, i.e.,

$$\lim_{G \rightarrow \infty} \mathbb{E}[\hat{L}(\theta)] = \mathbb{E}[L(\theta)]$$

Proof. Recall expectation is linear wrt to the sum and scalars, ,

$$\begin{aligned} \mathbb{E}[\hat{L}(\theta)] &= \frac{1}{G} \mathbb{E} \left[\left\langle \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, A_i \right\rangle \right] \\ &= \frac{1}{G} \cdot \sum_i^G \mathbb{E} \left[\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i \right] \end{aligned}$$

For a finite G , these advantages are dependent on other outputs o_j because they couple sampled outputs together, i.e., A_i is a random variable that changes based on the other outputs sampled in a specific batch. Thus,

$$\mathbb{E} \left[\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i \right] \neq \mathbb{E}[L(\theta)]$$

Next, observe by the triangle inequality,

$$|\hat{L}(\theta)| = \left| \frac{1}{G} \left\langle \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, A_i \right\rangle \right| \leq \frac{1}{G} \left\langle \left| \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} \right|, |A_i| \right\rangle$$

The policy ratios can only generate a maximum number of T tokens from a fixed vocabulary size in each output. Since every sequence we generate has a softmax value, then $0 < \pi_{\theta_{old}}(o_i|q)$, i.e., every token in the vocabulary has a non-zero probability of appearing, then

$$\left| \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} \right| \leq M_\pi < \infty$$

for some fixed M_π .

Assuming the reward space is compact (closed and bounded, i.e., contains all of its limit points since we're dealing with a computer-based task that has space limitations. No reward is technically infinite otherwise the whole model training would fall apart.), then

$$|r_i| \leq M_r < \infty$$

for some fixed M_r .

For A_i , observe recall

$$|A_i| = \left| \frac{r_i - \bar{r}_G}{\hat{\sigma}_G + \varepsilon} \right| = \frac{|r_i - \bar{r}_G|}{\hat{\sigma}_G + \varepsilon}$$

Then by the triangle inequality and transitive property,

$$|A_i| = \frac{|r_i - \bar{r}_G|}{\hat{\sigma}_G + \varepsilon} \leq \frac{|r_i| + |\bar{r}_G|}{\hat{\sigma}_G + \varepsilon}$$

The rewards being bounded implies their sample mean is also finitely bounded (by the same value as well). For $\hat{\sigma}_G$, recall its definition

$$\hat{\sigma}_G^2 = \frac{1}{G} \sum_{j=1}^G (r_j - \bar{r}_G)^2 \implies \hat{\sigma}_G = \frac{1}{\sqrt{G}} \sqrt{\sum_{j=1}^G (r_j - \bar{r}_G)^2}$$

Since squares are non-negative, the sum of squares is non-negative. It is clear $\hat{\sigma}_G$ is also non-negative. Since $0 < \varepsilon$ by definition, then

$$0 < \varepsilon \leq \hat{\sigma}_G + \varepsilon \implies \frac{1}{\hat{\sigma}_G + \varepsilon} \leq \frac{1}{\varepsilon}$$

Thus,

$$|A_i| \leq \frac{|r_i| + |\bar{r}_G|}{\hat{\sigma}_G + \varepsilon} \leq \frac{M_r + M_r}{\varepsilon} = \frac{2M_r}{\varepsilon}$$

By the Law of Large numbers, as G approaches infinity, the sample mean and sample standard deviation converge a.e. to the true population mean and standard deviation of the reward distribution under $\pi_{\theta_{old}}$, i.e.,

$$\lim_{G \rightarrow \infty} \overline{\{r_i\}_{i=1}^G} = \lim_{G \rightarrow \infty} \mu_G \xrightarrow{a.e.} \mu$$

and

$$\lim_{G \rightarrow \infty} \text{std}(\{r_i\}_{i=1}^G) = \lim_{G \rightarrow \infty} \sigma_G \xrightarrow{a.e.} \sigma$$

so we can define

$$A_i^{(G)} \xrightarrow{a.e.} A_i^* := \frac{r_i - \mu}{\sigma + \varepsilon}$$

which satisfies the same bound as A_i .

The A_i and A_i^* are bounded above by a function with a finite integral. By the Dominated Convergence Theorem, observe

$$\begin{aligned} \lim_{G \rightarrow \infty} \mathbb{E} [\hat{L}_G(\theta)] &= \lim_{G \rightarrow \infty} \frac{1}{G} \mathbb{E} \left[\left\langle \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, A_i \right\rangle \right] \\ &= \frac{1}{G} \mathbb{E} \left[\lim_{G \rightarrow \infty} \left\langle \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, A_i \right\rangle \right] \\ &= \frac{1}{G} \mathbb{E} \left[\left\langle \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, A_i^* \right\rangle \right] \\ &= \frac{1}{G} \cdot \sum_i^G \mathbb{E} \left[\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, A_i^* \right] \end{aligned}$$

Since the outputs o_i are sampled *iid* from $\pi_{\theta_{old}}$ every single term under the summation evaluates to the exact same true population objective $\mathbb{E}[L(\theta)]$, i.e.,

$$\mathbb{E}\left[\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, A_i^*\right] = \mathbb{E}[L(\theta)]$$

Finally,

$$\lim_{G \rightarrow \infty} \mathbb{E}[\hat{L}_G(\theta)] = \frac{1}{G} \cdot \sum_i^G \mathbb{E}\left[\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, A_i^*\right] = \frac{1}{G} \cdot G \mathbb{E}[L(\theta)] = \mathbb{E}[L(\theta)]$$

which satisfies the definition of an unbiased estimator. $\square$

Remember, LLMs don't just talk, they return a probability distribution token-by-token.

We currently have ratios of outputs, but these can be broken down to become ratios of tokens to get a new objective function.

just show me explicitly how the ratio of outputs becomes the ratio of tokens

You don't want to take big steps and stay within a range. So we clip because dividing probabilities can result in really large numbers. Since the ratio has range $(0, \infty)$, the clip keeps it by itself.

$$\begin{aligned} clip(x, \varepsilon) &= \begin{cases} 1 - \varepsilon & x < 1 - \varepsilon \\ x & 1 - \varepsilon \leq x \leq 1 + \varepsilon \\ 1 + \varepsilon & 1 + \varepsilon < x \end{cases} \\ &= \min(\max(x, 1 - \varepsilon), 1 + \varepsilon) \\ &= \max(\min(x, 1 + \varepsilon), 1 - \varepsilon) \end{aligned}$$

11 KL divergence

Use the base reference model as a reference and punish outputs that are too far.

$$\mathbb{D}_{KL} = p_i \log\left(\log\left(\frac{q_i}{p_i}\right)\right)$$

This assumes we trust our base model. This is not distance because it's not symmetric. In this case, we actually compare the probability distributions to test for divergence. It is not commutative (since there is one q_i and two p_i terms), because sampling probabilities have to make sense and not be surprising. For example, let our sample space consist of salad elements like lettuce, tomatoes, onions, and dressing. If one sampling distribution represents a salad, which is a spread of the entire sample space, and let the other sampling distribution represent a sack of tomatoes, which is a spread of only tomatoes from the sample space. Sampling from the salad can yield a tomato, but sampling from a sack of tomatoes cannot give you lettuce. Thus, the *KL*-divergences are

$$\mathbb{D}_{KL}(\text{salad}, \text{tomatoes}) = \infty$$

$$\mathbb{D}_{KL}(\text{tomatoes}, \text{salad}) = 1.6$$

Remark 11.1. you can rewrite KL-divergence as the difference between the cross-entropy of p & q (how close they are) and the entropy of p (how crazy p is, how predictable)

So in simplified terms the objective function looks like

$$\frac{1}{G} \sum \frac{1}{|o_i|} \sum \left(clip\left(\frac{\pi_{\theta_{new}} \hat{A}_i}{\pi_{\theta_{old}}}\right) - \mathbb{D}_{KL} \right)$$

Generate answers, each answer has many steps. Answers with good scores get increased probabilities, clip to not make big steps, and try not to veer off from the original model.

In the RL Training Process — do you freeze some of the layers and only fine tune only some of the layers with the gradients? You would use smart methods to train a model. A model with a trillion weights is

crazy. You can use the Lora method to factor the weight space into chunks and explore in smaller spaces. LORA are adapter matrices

<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-reasoning-llms>

we cannot use GRPO for anything else other than something where rewards are verifiable?

Use keynote to make the slides.

12 References

- <https://medium.com/@abhirugupta123/deep-dive-into-grpo-03826359836c>
- <https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-reasoning-llms> - process reward models
- <https://www.youtube.com/watch?v=gSPRxJLxIHA> - Jay's keynote presentation tutorial
- <https://research.nvidia.com/labs/nemotron/nemotron-cascade/> - easy problems first then cascading hard problems